

# SYNTH-04: Private Synthetic Locations

---

**Series:** SYNTH — Synthetic Monitoring | **Notebook:** 4 of 6 | **Created:** December 2025 | **Last Updated:** 09/28/2026

## Monitoring Internal Applications from Your Infrastructure

---

This notebook covers deploying and managing private synthetic locations (ActiveGates) for monitoring internal applications, APIs, and services not accessible from the public internet.

---

## Table of Contents

---

1. [Why Private Locations?](#)
  2. [Architecture](#)
  3. [Synthetic-Enabled ActiveGate](#)
  4. [Deployment Options](#)
  5. [Configuration](#)
  6. [Monitoring Private Location Health](#)
  7. [Troubleshooting](#)
- 

## Prerequisites

---

-  Access to a Dynatrace environment with Synthetic Monitoring

-  Completed SYNTH-01 through SYNTH-03
-  Infrastructure access to deploy ActiveGate (for setup)

## 1. Why Private Locations?

---

### Public vs Private Locations

| Aspect      | Public Locations     | Private Locations   |
|-------------|----------------------|---------------------|
| Hosting     | Dynatrace cloud      | Your infrastructure |
| Access      | Public internet only | Internal networks   |
| Maintenance | Managed by Dynatrace | Managed by you      |
| Security    | External perspective | Behind firewall     |
| Latency     | Varies by region     | Local network       |

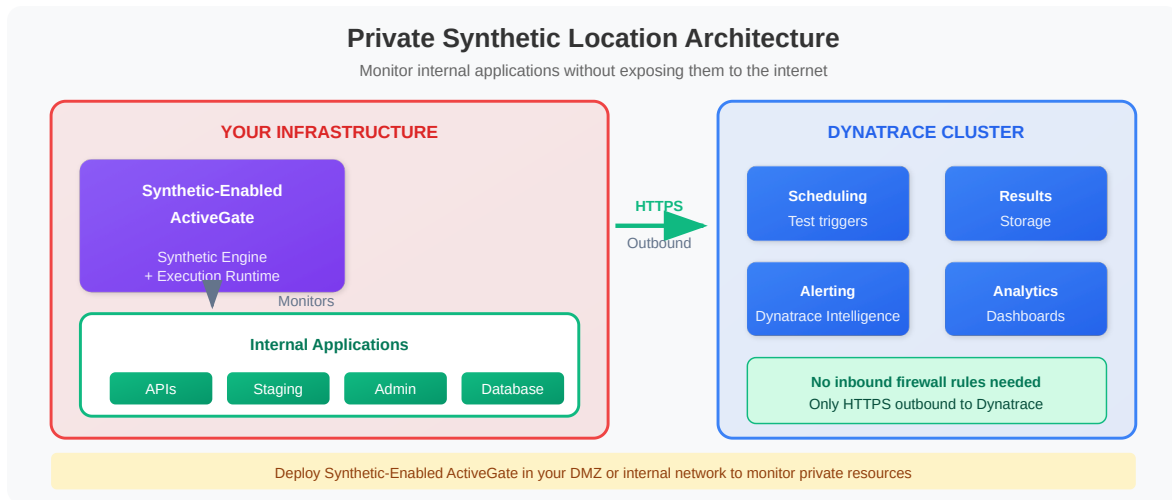
### Use Cases for Private Locations

| Scenario            | Description                       |
|---------------------|-----------------------------------|
| Internal APIs       | Services not exposed to internet  |
| VPN-only apps       | Applications requiring VPN access |
| Pre-production      | Staging/dev environments          |
| Security compliance | Data must not leave network       |
| Low latency testing | Local performance baselines       |
| Isolated networks   | Air-gapped or restricted networks |

## 2. Architecture

---

Private synthetic locations use ActiveGates deployed within your infrastructure to execute monitors against internal applications:



## Key Points

- **Outbound only:** ActiveGate initiates all connections
- **No inbound ports:** No firewall changes for external access
- **Local execution:** Tests run inside your network
- **Results upload:** Only metrics/results sent to Dynatrace

## 3. Synthetic-Enabled ActiveGate

### ActiveGate Capabilities

ActiveGate can serve multiple purposes:

| Capability        | Description                |
|-------------------|----------------------------|
| <b>Synthetic</b>  | Private synthetic location |
| <b>Metrics</b>    | Metric ingestion endpoint  |
| <b>Logs</b>       | Log ingestion endpoint     |
| <b>API</b>        | Cluster API proxy          |
| <b>Extensions</b> | Extension execution        |

## Synthetic Engine Requirements

| Resource         | Minimum (XS node with browser support)                                                   | Guidance                                                                                                                   |
|------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| CPU              | 2 vCPU                                                                                   | 4 vCPU for an S node — pick the node size from the sizing guide's executions-per-hour limits                               |
| RAM              | 4 GB                                                                                     | At least 8 GB                                                                                                              |
| Disk             | 20 GB free                                                                               | At least 25 GB free                                                                                                        |
| Network          | HTTPS outbound                                                                           | Low latency to targets                                                                                                     |
| Operating system | A Linux or Windows Server release currently supported for a synthetic-enabled ActiveGate | Track the ActiveGate OS support matrix — supported releases are versioned with the <b>ActiveGate</b> , not with the tenant |

The 8 GB / 25 GB guidance is the requirements page's own: *"we recommend having at least 8 GB of RAM and 25 GB of free disk space."* Size is also sticky — *"To change the size of a Synthetic-enabled ActiveGate, for example, after upgrading from size S to meet size M requirements, you must uninstall and reinstall it."*

**Windows Server 2025** is supported for private synthetic locations. The OS option first appeared for synthetic-capable ActiveGates with **ActiveGate 1.341**; **SaaS 1.344** (release notes published 07/27/2026) states it in the private-location context directly: *"You can now install on Windows Server 2025 hosts for private synthetic locations."* The documentation statement is firm — published docs are live regardless of which sprint your tenant is on — but the capability ships in the **ActiveGate installer**, so confirm the ActiveGate build you are about to deploy carries it before committing a Windows Server 2025 rollout plan.

ActiveGate **1.343** (published 07/15/2026, rollout from 07/28/2026) additionally adds OS support for **Oracle Linux 9.8 and 10.2** and **Rocky Linux 9.8 and 10.2**, and flags a support-discontinuation wave running **November 2026 through January 2027** covering older Red Hat Enterprise Linux, Ubuntu 16.04, Oracle Linux, Rocky Linux, and Amazon Linux 2 builds across several architectures. Audit your private-location fleet against that list before it starts.

## Browser Monitor Requirements

For browser monitors, additional requirements:

- A Chromium-family browser, provided through the ActiveGate rather than installed by you

— on **Windows** the installer package includes Chrome for Testing; on **Linux** the installer downloads the browser and its dependencies. Its version tracks your **ActiveGate fleet** (and, on Linux, the location's browser auto-update switch — see below), **not your tenant version**

- Write access to `/tmp` — the browser's dependencies, including `xvfb`, are installed with it and use `/tmp` (no separate display server to install)
- Additional RAM for browser instances

#### Browser baseline by ActiveGate version:

| ActiveGate                                                                                                          | RHEL 9.7,<br>Rocky Linux<br>9.8 | Ubuntu 20.04 / 22.04<br>/ 24.04, Amazon<br>Linux 2023, Oracle<br>Linux 9.7 | Windows<br>(bundled in<br>the installer) |
|---------------------------------------------------------------------------------------------------------------------|---------------------------------|----------------------------------------------------------------------------|------------------------------------------|
| <b>1.343</b> (published 07/15/2026,<br>rollout from 07/28/2026)                                                     | Chromium<br>150                 | Chrome for Testing<br>150                                                  | Chrome for<br>Testing 150                |
| <b>1.345</b> (rollout from<br>08/25/2026)                                                                           | Chromium<br>151                 | Chrome for Testing<br>151                                                  | Chrome for<br>Testing 151                |
| <b>1.347</b> (published 09/10/2026<br>as a planned release, <b>staged<br/>rollout planned from<br/>09/22/2026</b> ) | Chromium<br>152                 | Chrome for Testing<br>152                                                  | Chrome for<br>Testing 152                |

ActiveGate 1.347 is forthcoming — verify it has reached your synthetic ActiveGates before relying on Chromium 152; until then, the 1.345 row is the working baseline for upgraded fleets, and the 1.343 row for the rest.

**How the browser is updated.** The browser moves with ActiveGate and Synthetic engine updates, not on a schedule of its own: *"The browser autoupdate takes place during manual as well as automatic ActiveGate and Synthetic engine updates."* On **Linux** this is controlled per private location by the **Enable Chrome(-ium) auto-update** switch, which is on by default (*"the browser autoupdate is turned on by default for locations with Linux-based ActiveGates"*). Turn it off only to hold a specific browser version or for offline environments — and then update the browser manually on every ActiveGate in the location. On **Windows** there is no switch: *"on Windows-based ActiveGates, the browser is always updated during Synthetic engine updates."* So the lever for holding a browser version is that per-location switch, **not** ActiveGate auto-update. Two support rules bound how far a location can drift: Dynatrace *"supports browser versions that are no more than two versions behind the latest Dynatrace-supported version for a specific ActiveGate"*

release", and "We strongly recommend updating all ActiveGates per location to the same version."

Because the browser can change whenever an ActiveGate or engine update lands, re-check browser monitors after an update before treating new failures as application regressions.

**Sources:** [ActiveGate 1.343 \(DT docs\)](#) — "Chrome for Testing 150 is bundled with the Windows ActiveGate installer."; [ActiveGate 1.345 \(DT docs\)](#) — "Chrome for Testing 151 is bundled with the Windows ActiveGate installer."; [ActiveGate 1.347 \(DT docs\)](#) — "Chromium 152 is the latest supported version for Synthetic-enabled ActiveGate" and "Chrome for Testing 152 is bundled with the Windows ActiveGate installer."; [Requirements for private Synthetic locations \(DT docs\)](#) — "Its dependencies, including xvfb, utilize /tmp"; [Manage private Synthetic locations \(DT docs\)](#) — the auto-update, Windows, two-versions-behind and same-version quotes above.

Two things follow from the browser being tied to the ActiveGate. First, an ActiveGate that has not been updated — or a Linux location with browser auto-update turned off — is still executing clickpaths in the older browser however current the tenant is — so when the same clickpath behaves differently at two private locations, compare **ActiveGate versions before** suspecting the application:

```
smartscapeNodes "ACTIVEGATE"
| fields name, dt.active_gate.version, dt.active_gate.group.name, os.type,
os.version
| sort dt.active_gate.version asc, name asc
```

Second, the **1.331 floor stated below is a minimum, not a target**. It is the version that lets users whose access is scoped by security context see private-location results; the ActiveGate build (and, on Linux, the location's browser auto-update switch) determines which browser your clickpaths actually run in. Both matter, for different reasons. (For the ActiveGate update-management model on SaaS — auto-update windows, version pinning, staged fleet upgrades — see FAQ-05; the browser itself is governed by the per-location switch above.)

**Security context and ActiveGate 1.331 (SaaS 1.343, July 2026 — staged tenant rollout from mid-July).** SaaS 1.343 migrates the management zones of each synthetic **monitor** to security-context values: "Dynatrace performs a one-time migration of the management zone each synthetic monitor belongs to, mapping them

to security context values with the same name on that monitor." It runs once — "Updating or creating management zones and monitors won't be synchronized." On private locations, ActiveGate **1.331+** is what makes those results scopable: "Earlier versions do not enrich the metrics and events produced by Synthetic monitor executions with the monitor's security context value. Without this enrichment, IAM policies scoped to security contexts cannot grant access to execution data." An older ActiveGate keeps executing monitors; users whose access is scoped by security context just cannot see its results. Upgrade any synthetic AG below 1.331 before you move users to security-context-scoped policies, and review monitor access once the migration reaches your tenant — the security-context model replaces MZ-based scoping (the MZ2POL series covers the broader management-zone migration). SaaS 1.343 also adds IAM-based access control for Synthetic Monitoring and API support for assigning security context to synthetic monitors.

**Sources:** [SaaS 1.343 \(DT docs\)](#), [Access control for Synthetic \(DT docs\)](#).

## 4. Deployment Options

### Option 1: Linux/Windows Installer

Download the installer from **Settings** → **Deployment status** → **ActiveGate**, run it with the synthetic flag, then check the service:

```
sudo /bin/sh Dynatrace-ActiveGate-Linux-x86-*.sh \
  --enable-synthetic

sudo systemctl status dynatracegateway
```

### Option 2: Container Deployment

`docker-compose.yml` :

```

version: '3'
services:
  activegate:
    image: dynatrace/dynatrace-activegate:latest
    environment:
      - DT_TENANT=abc12345
      - DT_CAPABILITIES=synthetic
      - DT_API_TOKEN=${DT_API_TOKEN}
    volumes:
      - activegate-data:/var/lib/dynatrace/gateway
    ports:
      - "9999:9999"

```

### Option 3: Kubernetes / OpenShift (containerized locations)

Containerized private locations are deployed from **templates the Synthetic app generates**, not from a DynaKube custom resource — there is no `synthetic-monitoring` ActiveGate capability to add to a DynaKube. In **Synthetic** → **Private locations**, create a containerized location and select **Download synthetic.yaml** (*"This is the location template file."*), then deploy the metric adapter from its own downloaded template (*"This is the template file for the Synthetic metric adapter."*). Apply both with the `kubectl` commands the UI generates.

 **Latest Dynatrace — metric adapter (Sprint 1.339):** the variable belongs on the **Synthetic metric adapter** deployment, not on the location pod: *"Modify the adapter deployment. Set the environment variable METRIC\_3RD\_GEN\_ENABLED to "true"."* The same migration requires you to *"Set the environment variable BASE\_URL to match the Latest Dynatrace URL."* and to *"Modify synthetic location deployments—change the target metric in the Horizontal Pod Autoscaler definition."* Re-download the templates from **Synthetic** → **Private locations**, or patch existing manifests per those steps, and schedule the change through change management.

**Sources:** [Containerized private Synthetic locations on Kubernetes \(DT docs\)](#).

```

// List all synthetic locations, public and private
// PREFERRED -- Smartscape. The SYNTHETIC_LOCATION node exposes location_type
DIRECTLY,
// which makes the old "identify private locations by naming convention"
workaround
// obsolete. `stage` is the lifecycle marker (GA / BETA / DEPRECATED /
COMING_SOON) --
// worth scanning for locations Dynatrace is retiring under you.
smartscapeNodes "SYNTHETIC_LOCATION"
| fields name, location_type, stage, cloud.provider, geo.country.name,
geo.city.name, id_classic
| sort location_type asc, name asc
| limit 50

// To list only private locations, add:
// | filter location_type == "PRIVATE"
//
// VERIFY THE LITERAL FIRST. location_type is confirmed present and populated,
but the
// validation tenant holds only PUBLIC locations (93 of them, no private
ones), so the
// literal "PRIVATE" is INFERRED from the field's public/private semantics
rather than
// observed. Run the summarize query in the next cell against YOUR tenant to
see the
// exact literals it returns before hard-coding one into a dashboard, alert,
or
// automation filter -- a wrong literal returns zero rows, not an error.

// FALLBACK (classic surface -- deprecated in DQL, supported for as long as
Dynatrace
// Classic is supported, and genuinely lacks these fields.
// This is why the previous revision of this notebook said
type/status/city/countryCode
// "are not available": on the classic entity that was true, and remains
true.)
// fetch dt.entity.synthetic_location
// | fields id, entity.name
// | sort entity.name asc
// | limit 50

```

```
// Count synthetic locations by type and lifecycle stage
// Run THIS FIRST -- it tells you the exact location_type literals your tenant
returns,
// which is what the previous cell's filter depends on.
smartscapeNodes "SYNTHETIC_LOCATION"
| summarize locations = count(), by:{location_type, stage}
| sort location_type asc, locations desc

// FALLBACK (classic surface, deprecated in DQL) -- can only produce a grand
total, because the classic
// entity has no type field at all. That limitation is exactly what the old
// naming-convention workaround was compensating for.
// fetch dt.entity.synthetic_location
// | summarize {count = count()}
// | fields count
```

## 5. Configuration

### Creating a Private Location

1. **Deploy ActiveGate** with synthetic capability
2. **Navigate to: Synthetic → Private locations** (or, in Settings Classic, **Settings → Web and mobile monitoring → Private Synthetic locations**)
3. **Create location:** Name, description, geographic info
4. **Assign ActiveGates:** Select which ActiveGates serve this location

### Location Settings

| Setting                   | Description                                                               |
|---------------------------|---------------------------------------------------------------------------|
| <b>Name</b>               | Descriptive location name                                                 |
| <b>Latitude/Longitude</b> | Geographic coordinates                                                    |
| <b>City/Region</b>        | Location metadata                                                         |
| <b>ActiveGate nodes</b>   | Assigned ActiveGates — one ActiveGate cannot serve more than one location |

| Setting                                | Description                                                                                                                                                       |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Enable Chrome(-ium) auto-update</b> | Linux-based locations; on by default. Updates the browser during ActiveGate and Synthetic engine updates (see § 3). API property: <code>autoUpdateChromium</code> |

## High Availability

For production workloads:

- Deploy 2+ ActiveGates per location
- Keep every ActiveGate in a location on the same version
- Distribute across availability zones
- Load balancing automatic

## Declarative Creation via the API

The UI flow above is the interactive path. For repeatable, reviewable, source-controlled private-location definitions, use the Environment API v2 endpoint:

```
POST /api/v2/synthetic/locations
```

**API 1.344** (published 07/15/2026, **staged rollout from 07/29/2026**) adds three properties to the `PrivateSyntheticLocation` request schema of `POST /synthetic/locations` — `minActiveGateCount`, `maxActiveGateCount`, and `nodeSize`. All three are **containerized-location** properties: the POST reference describes each as a *"Containerized location property"* that is *"required for a Kubernetes location"*. They are the API counterparts of the fields the UI asks for when you create a Kubernetes or OpenShift location — the minimum and maximum number of ActiveGates the horizontal pod autoscaler works between, and the ActiveGate node size. They do not size a standard Linux/Windows location, whose ActiveGate size is set by the host it is installed on (see § 3).

| Property                        | Purpose                                                                                                                             |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>minActiveGateCount</code> | Minimum number of ActiveGates — the autoscaler's lower bound. The docs recommend <i>"a minimum of two ActiveGates per location"</i> |
| <code>maxActiveGateCount</code> | Maximum number of ActiveGates — caps scale-out                                                                                      |

| Property              | Purpose                                                                                                                                                                                                                                                                                         |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>nodeSize</code> | ActiveGate node size: <code>XS</code> , <code>S</code> or <code>M</code> ( <i>"The node size L is not supported in containerized locations."</i> ). Choose deliberately — <i>"Once specified, ActiveGate size for a location can't be changed because persistent storage can't be resized."</i> |

Because the rollout is staged, confirm the tenant you are automating accepts the three properties before relying on them; until it does, creating the containerized location in the UI remains the working path.

**Authentication boundary (carried over from SYNTH-01):** synthetic monitor and location management still requires an **API token**, not a platform token — including through the Terraform provider (see AUTOM-04 § Authentication Boundary). Do not design a private-location automation path around platform-token auth.

#### Sources:

- [Dynatrace API release notes 1.344 \(DT docs\)](#) — `POST /synthetic/locations` : changed `PrivateSyntheticLocation` schema, added `maxActiveGateCount` , `minActiveGateCount` , `nodeSize`
- [Synthetic locations API \(DT docs\)](#)
- [POST a location \(DT docs\)](#) — *"Containerized location property. The minimum number of ActiveGates deployed for the location (required for a Kubernetes location)."*
- [Containerized private Synthetic locations on Kubernetes \(DT docs\)](#) — *"We recommend the S ActiveGate size and a minimum of two ActiveGates per location."*
- [Private locations in Synthetic \(DT docs\)](#) — *"We recommend using at least two ActiveGates for a location. You can't use one ActiveGate for multiple locations."*
- [Manage private Synthetic locations \(DT docs\)](#) — *"We strongly recommend updating all ActiveGates per location to the same version."*

```
// Monitor executions by location
// Private locations typically have custom names (not city names like "N.
Virginia")
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| summarize {
    executions = count(),
    availability_pct = round(countIf(result.state == "SUCCESS") * 100.0 /
count(), decimals: 2)
}, by: {monitor.name, dt.entity.synthetic_location}
| fieldsAdd location = entityName(dt.entity.synthetic_location)
| sort executions desc
| limit 30
```

## 6. Monitoring Private Location Health

### ActiveGate Health Indicators

| Metric              | Description           | Alert Threshold |
|---------------------|-----------------------|-----------------|
| <b>Connectivity</b> | Connection to cluster | Any disconnect  |
| <b>CPU Usage</b>    | Processing load       | > 80% sustained |
| <b>Memory</b>       | RAM utilization       | > 80%           |
| <b>Disk</b>         | Storage usage         | > 80%           |
| <b>Queue Depth</b>  | Pending executions    | > 100           |

```
// Synthetic location health status (1 = healthy)
// Surfaces location/ActiveGate availability independent of monitor results
timeseries health = avg(dt.synthetic.location.health_status),
    from: now() - 24h, interval: 1h, by: {dt.entity.synthetic_location}
```

```
// Location availability over time (last 7 days)
fetch dt.synthetic.events, from: now() - 7d
| filter endsWith(event.type, "_monitor_execution")
| makeTimeseries {
    success_count = countIf(result.state == "SUCCESS"),
    total_count = count()
}, interval: 1h, by: {dt.entity.synthetic_location}
| fieldsAdd availability_pct = success_count[] * 100.0 / total_count[]
```

```
// Execution count by location
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| summarize {
    total_executions = count(),
    failed_executions = countIf(result.state == "FAIL"),
    unique_monitors = countDistinct(dt.synthetic.monitor.id)
}, by: {dt.entity.synthetic_location}
| fieldsAdd failure_rate = round((failed_executions * 100.0) /
total_executions, decimals: 2)
| fieldsAdd location = entityName(dt.entity.synthetic_location)
| sort total_executions desc
```

## 7. Troubleshooting

### Common Issues

| Issue                      | Symptoms             | Resolution                     |
|----------------------------|----------------------|--------------------------------|
| <b>No connectivity</b>     | Location offline     | Check ActiveGate logs, network |
| <b>SSL errors</b>          | Certificate failures | Install CA certs on ActiveGate |
| <b>Timeouts</b>            | All tests fail       | Check network routes, DNS      |
| <b>Resource exhaustion</b> | Slow/failed tests    | Scale ActiveGate resources     |
| <b>Browser issues</b>      | Clickpath failures   | Check Chrome/display config    |

### ActiveGate Logs

Linux log location and key log files:

```
/var/log/dynatrace/gateway/  
gateway.log           Main gateway log  
synthetic.log         Synthetic execution log  
connection.log        Cluster connectivity
```

## Network Verification

```
# Test connectivity to target  
curl -v https://internal-api.company.com/health  
  
# DNS resolution  
nslookup internal-api.company.com  
  
# Test Dynatrace connectivity  
curl -v https://&lt;tenant&gt;.live.dynatrace.com/api/v1/time
```

```
// Identify failing monitors by location  
// To filter for specific private locations, add: | filter  
dt.entity.synthetic_location == "SYNTHETIC_LOCATION-..."  
fetch dt.synthetic.events, from: now() - 24h  
| filter endsWith(event.type, "_monitor_execution")  
| filter result.state == "FAIL"  
| summarize {  
    failure_count = count(),  
    last_failure = max(timestamp)  
}, by: {monitor.name, dt.entity.synthetic_location}  
| fieldsAdd location = entityName(dt.entity.synthetic_location)  
| sort failure_count desc  
| limit 20
```

---

## Summary

---

In this notebook, you learned:

- ✓ **Why private locations** - Internal apps, security, compliance
- ✓ **Architecture** - ActiveGate with synthetic engine
- ✓ **Deployment options** - Installer, container, Kubernetes templates (+ adapter `METRIC_3RD_GEN_ENABLED` for Latest tenants)
- ✓ **Configuration** - Creating and managing locations

✔ **Health monitoring** - `dt.synthetic.location.health_status` metric + execution results by location

✔ **Troubleshooting** - Common issues and resolution

---

## Next Steps

---

Continue to **SYNTH-05: Network Monitoring** to learn about synthetic network availability monitoring.

---

## References

---

- [Private synthetic locations \(DT docs\)](#)
  - [Synthetic architecture and communication \(DT docs\)](#)
  - [ActiveGate \(DT docs\)](#)
  - [Containerized private Synthetic locations on Kubernetes \(DT docs\)](#)
  - [Requirements for private Synthetic locations \(DT docs\)](#)
  - [Access control for Synthetic \(DT docs\)](#)
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*